

Report: Semantic Object Segmentation in Cultural Sites using PSPNet and OS-DCLGAN

Diego Martinez

July 2026

Contents

1	Introduzione alla Segmentazione Semantica	4
1.1	Definizione del Task e differenze con la Classificazione	4
1.2	Applicazioni nel dominio dei Beni Culturali (Cultural Heritage) .	4
1.3	Sfide principali: Sbilanciamento delle classi e complessità computazionale	5
2	Il Problema della Domain Adaptation	5
2.1	Definizione di Domain Shift: Dati Sintetici vs Dati Reali	5
2.2	Unsupervised vs Semi-Supervised Domain Adaptation	6
2.3	Il problema del "Cold Start" e il collasso delle predizioni	7
3	Architettura per la Segmentazione: PSPNet	7
3.1	Il backbone di estrazione feature (ResNet-50)	8
3.2	Il Pyramid Pooling Module (PPM)	8
3.3	Funzione di Costo: Cross Entropy Loss e pesi "Inverse Frequency" .	9
4	Traduzione di Immagini e Generative Adversarial Networks (GAN)	10
4.1	Basi delle GAN: Generatore e Discriminatore	10
4.2	Limiti della CycleGAN e introduzione all'apprendimento contrastivo (CUT)	10
4.3	Architettura OS-DCLGAN (One-Sided Domain Adaptation) . . .	11
4.4	PatchNCE Loss e modulo di attenzione spaziale (CBAM)	11
5	Metriche di Valutazione per la Segmentazione	12
5.1	Intersection over Union (IoU) e Mean IoU (mIoU)	12
5.2	Pixel Accuracy (PA) e Mean Pixel Accuracy (mPA)	12
5.3	Frequency Weighted Average Accuracy (FWAVACC)	13
5.4	La Matrice di Confusione	13
6	Segmentazione di Oggetti Culturali: Il Dataset EGO-CH-OBJ-SEG	13
6.1	Analisi esplorativa del Dataset (Sintetico e Reale)	13
6.2	Data Engineering: Risoluzione dell' "Exact RGB Matching" . . .	14
6.3	Tecniche di Data Augmentation e Preprocessing	14
7	Implementazione e Metodologia di Training	15
7.1	Architettura di Joint-Training (Training congiunto GAN + PSP-Net)	15
7.2	Stabilità Numerica: Risoluzione dei NaN, Gradient Clipping e Precisione FP32	15
7.3	La strategia di "Warm-up" per la salvaguardia del Segmentatore .	16

8	Esperimenti e Risultati	17
8.1	Setup dell'Infrastruttura e Limiti Computazionali	17
8.2	Analisi dei Risultati	17
8.2.1	Domain Adaptation Fully-Supervised (100% Dati Reali) .	19
8.2.2	Domain Adaptation Semi-Supervisionata (50% Dati Reali)	19
8.2.3	Unsupervised Domain Adaptation (0% Dati Reali)	19
8.3	Considerazioni sull'efficienza (OS-DCLGAN vs CycleGAN) . . .	20
9	Conclusioni	20
9.1	Considerazioni finali sull'efficacia dell'approccio	20
9.2	Sviluppi e lavori futuri	21

1 Introduzione alla Segmentazione Semantica

1.1 Definizione del Task e differenze con la Classificazione

Nell'ambito della Visione Artificiale (*Computer Vision*), l'estrazione di informazioni semantiche dalle immagini è affrontata attraverso task a diversi livelli di granularità.

Nel task di base, la **Classificazione delle Immagini**, l'obiettivo è assegnare una singola etichetta globale all'intera immagine (ad esempio, stabilire se in una foto è presente o meno una "Statua"). Un passo avanti è rappresentato dall'**Object Detection**, che non solo identifica la presenza di un oggetto, ma ne stima anche la posizione spaziale circoscrivendolo all'interno di un rettangolo delimitatore (*bounding box*).

Tuttavia, per una reale e profonda comprensione della scena, i *bounding box* risultano insufficienti, in quanto includono inevitabilmente porzioni di sfondo e non descrivono la forma esatta dell'oggetto. La **Segmentazione Semantica** supera questo limite ponendosi come un problema di *dense prediction* (predizione densa): l'obiettivo è assegnare un'etichetta di classe a ogni singolo pixel dell'immagine. Il risultato finale è una mappa di segmentazione ad alta risoluzione in cui ogni oggetto è perfettamente delineato nei suoi contorni esatti. In termini matematici, data un'immagine di dimensioni $H \times W \times 3$, una rete di segmentazione produce un tensore di output di dimensioni $H \times W \times C$, dove C rappresenta il numero totale delle classi previste dal dataset.

1.2 Applicazioni nel dominio dei Beni Culturali (Cultural Heritage)

La capacità di riconoscere a livello di pixel gli elementi di una scena apre scenari applicativi di immenso valore, specialmente nel dominio della preservazione e valorizzazione dei Beni Culturali (*Cultural Heritage*).

In ambienti complessi e affollati come musei, siti archeologici o gallerie d'arte, la segmentazione semantica è la tecnologia abilitante per lo sviluppo di:

- **Robotica Assistiva e Guide Autonome:** Robot in grado di navigare in sicurezza le sale museali, riconoscendo ostacoli, teche espositive e opere d'arte per interagire con i visitatori.
- **Realtà Aumentata (AR) e Wearable Devices:** Sistemi di visione egocentrica (indossati dall'utente, come smart glasses) capaci di evidenziare nel campo visivo dell'osservatore dettagli specifici di un reperto, sovrapponendo informazioni storiche direttamente sui contorni dell'oggetto reale.
- **Digitalizzazione e Restauro:** Strumenti automatici per l'isolamento e la catalogazione di opere d'arte da vasti archivi fotografici.

Il presente progetto si inserisce esattamente in questo contesto, utilizzando come riferimento il dataset EGO-CH-OBJ-SEG, focalizzato sul riconoscimento se-

mentico di 24 classi di oggetti culturali (quadri, sculture, teche, estintori, cartelli informativi) acquisiti da una prospettiva egocentrica.

1.3 Sfide principali: Sbilanciamento delle classi e complessità computazionale

Addestrare reti neurali profonde per la predizione densa comporta sfide ingegneristiche e matematiche notevolmente superiori rispetto alla semplice classificazione. Due, in particolare, rappresentano gli ostacoli critici per la convergenza dei modelli:

1. **Lo sbilanciamento estremo delle classi (*Class Imbalance*):** Nelle immagini reali e sintetiche di ambienti chiusi, classi strutturali come muri, pavimenti e soffitti occupano inevitabilmente la stragrande maggioranza dei pixel. Nel caso specifico del dataset analizzato, la classe "Background" (Classe 0) arriva a coprire oltre il 50% dei pixel totali disponibili, mentre classi minoritarie rappresentano frazioni infinitesimali dell'immagine. Se questo sbilanciamento non viene gestito matematicamente, la rete neurale tenderà a ottimizzare la propria *loss* adottando una scorciatoia statistica: collassare le predizioni assegnando tutti i pixel alla classe maggioritaria, perdendo la capacità di riconoscere qualsiasi altro oggetto.
2. **Complessità Computazionale e Perdita di Informazione Spaziale:** Rispetto alle reti di classificazione, che riducono progressivamente la dimensionalità spaziale delle *feature map* fino a ottenere un singolo vettore, le architetture per la segmentazione devono ripristinare la risoluzione originale per generare la maschera finale. Processare immagini ad alta risoluzione preservando il contesto globale e i dettagli locali richiede un dispendio enorme di memoria VRAM e architetture sofisticate in grado di aggregare informazioni a diverse scale, sfide che rendono complesso il bilanciamento dell'hardware a disposizione durante l'addestramento.

2 Il Problema della Domain Adaptation

2.1 Definizione di Domain Shift: Dati Sintetici vs Dati Reali

L'addestramento di modelli di segmentazione semantica richiede enormi quantità di dati etichettati a livello di pixel. Poiché l'annotazione manuale di scenari reali è un'operazione estremamente dispendiosa in termini di tempo e costi, la comunità scientifica fa sempre più affidamento su dataset sintetici. Sfruttando motori grafici (come Unity o Unreal Engine), è possibile generare un numero illimitato di immagini fotorealistiche con le relative maschere di segmentazione perfette (*ground truth*), generate automaticamente dal motore di rendering.

Tuttavia, un modello addestrato esclusivamente su immagini sintetiche subisce un drastico calo di performance quando viene testato su immagini reali. Questo

fenomeno è noto come **Domain Shift** (o *Domain Gap*). Dal punto di vista statistico, il Domain Shift si verifica quando la distribuzione di probabilità marginale dei dati sorgente (Sintetici, $P_S(X)$) è diversa dalla distribuzione dei dati target (Reali, $P_T(X)$), sebbene lo spazio delle etichette Y rimanga invariato.

Visivamente, questa discrepanza è causata da differenze intrinseche nella composizione delle immagini:

- **Fattori di illuminazione:** Le ombre, i riflessi e l'esposizione alla luce nel mondo reale sono molto complessi da simulare in modo perfetto.
- **Texture e Materiali:** Le superfici reali presentano imperfezioni, usura e variazioni di colore (*noise*) assenti nei modelli 3D.
- **Sensori di acquisizione:** Le fotocamere reali, specialmente in ottica egocentrica, introducono sfocature da movimento (*motion blur*), aberrazioni cromatiche e rumore del sensore.

L'obiettivo della **Domain Adaptation (DA)** è mitigare questo divario, trasferendo la conoscenza acquisita dal dominio sorgente (Sintetico) al dominio target (Reale).

2.2 Unsupervised vs Semi-Supervised Domain Adaptation

A seconda della disponibilità di etichette nel dominio target, le tecniche di Domain Adaptation si dividono in due paradigmi principali, entrambi esplorati all'interno di questo progetto:

1. **Unsupervised Domain Adaptation (UDA):** In questo scenario, si dispone di un dataset sorgente completamente etichettato e di un dataset target costituito esclusivamente da immagini non etichettate. L'obiettivo è allineare le distribuzioni dei due domini nello spazio delle *feature*. In questo progetto, l'UDA è stata affrontata sfruttando reti generative avversarie (GAN) per tradurre lo stile delle immagini sintetiche in quello reale, costringendo il segmentatore a generalizzare su un dominio "ibrido" senza mai avere accesso alle vere maschere reali. Questo paradigma incarna l'obiettivo ultimo della ricerca: azzerare completamente i costi di annotazione manuale, dimostrando che è possibile ottenere risultati di segmentazione accettabili affidandosi unicamente a dati generati sinteticamente.
2. **Semi-Supervised Domain Adaptation (SSDA):** Quando è disponibile una frazione limitata di dati target etichettati (es. 10%, 25% o 50% del dataset reale), ci si trova in un regime semi-supervisionato. L'integrazione di questi dati fornisce un segnale forte alla rete (tramite la *Supervised Loss*), guidandola nella corretta mappatura dei confini degli oggetti complessi che la sola traduzione di stile faticerebbe a risolvere. In un'ottica di ricerca sperimentale, l'SSDA rappresenta il compromesso strategico ideale: ridurre progressivamente la dipendenza dai dati reali per minimizzare il

dispendio di tempo e risorse umane, mantenendo al contempo prestazioni e correttezza visiva di alto livello.

2.3 Il problema del "Cold Start" e il collasso delle predizioni

L'addestramento congiunto (*Joint-Training end-to-end*) di un modello generativo (GAN) e di un modello di segmentazione (PSPNet) introduce sfide teoriche e pratiche considerevoli, note in letteratura come il problema del **Cold Start** (partenza a freddo).

Durante le primissime epoche di addestramento, il Generatore della GAN non ha ancora appreso come mappare lo stile sintetico in quello reale, producendo output visivi che sono essenzialmente rumore casuale o artefatti incomprensibili. In un'architettura *end-to-end*, queste immagini corrotte vengono passate direttamente alla rete di segmentazione, alla quale viene richiesto di calcolare una *Cross Entropy Loss* rispetto alle maschere perfette.

Questo processo innesca due criticità fatali per la rete:

1. **Il collasso statistico (*Background Collapse*):** Ricevendo input privi di senso, la rete di segmentazione riceve gradienti di errore elevatissimi. Per minimizzare la *Loss* e "sopravvivere", il modello impara rapidamente una scorciatoia locale: predire l'intera immagine come "Sfondo" (*Background*), dato che questa classe domina statisticamente la maggior parte dei pixel del dataset (spesso oltre il 50%). Una volta che i pesi collassano su questa soluzione costante, l'apprendimento delle *feature* spaziali per le altre classi si arresta in modo irreversibile.
2. **Instabilità numerica e divergenza (*NaN Explosion*):** Le reti avversarie e i moduli di attenzione basati su apprendimento contrastivo (come la *PatchNCE Loss*) dipendono fortemente da calcoli logaritmici su distribuzioni di probabilità. Output generativi di scarsa qualità nelle fasi iniziali possono portare il termine dell'attenzione a zero. Operazioni come $-\log(0)$, combinate con i limiti di rappresentazione della *Mixed Precision* (FP16), generano valori infiniti (*Inf*) o non numerici (*NaN*), causando la corruzione immediata dei pesi del modello.

Affrontare e risolvere queste dinamiche, tramite strategie architetturali e accorgimenti ingegneristici nella pipeline di addestramento, costituisce uno dei contributi fondamentali del lavoro svolto in questo progetto.

3 Architettura per la Segmentazione: PSPNet

Per affrontare la scomposizione densa della scena, il progetto adotta la **Pyramid Scene Parsing Network (PSPNet)**. L'efficacia di questa architettura risiede nella sua capacità di mitigare uno dei difetti storici delle *Fully Convolutional Networks* (FCN): la mancanza di comprensione del contesto globale. Ad

esempio, riconoscere una barca su un fiume è semanticamente diverso dal riconoscere una forma simile parcheggiata su una strada; la PSPNet è progettata per "osservare" l'intera scena prima di classificare il singolo pixel, sfruttando un modulo di *pooling* a piramide.

L'architettura implementata in questo lavoro si compone di due stadi fondamentali: un estrattore di feature (*backbone*) e un modulo di aggregazione contestuale.

3.1 Il backbone di estrazione feature (ResNet-50)

La fase iniziale di elaborazione dell'immagine è affidata a una rete convoluzionale profonda, specificamente una **ResNet-50**, pre-addestrata su ampi dataset generici (es. ImageNet) per garantire una solida estrazione di feature di base (bordi, texture, gradienti di colore).

Tuttavia, l'impiego diretto di una ResNet tradizionale per la segmentazione comporterebbe una perdita drastica della risoluzione spaziale. A causa dei ripetuti layer di *pooling* e dei *stride* convoluzionali, l'output finale di una ResNet standard ha una risoluzione pari a $1/32$ dell'immagine di input, rendendo impossibile recuperare i contorni precisi di oggetti piccoli.

Per preservare i dettagli spaziali, l'architettura della PSPNet modifica gli ultimi due stadi della ResNet (chiamati comunemente **layer3** e **layer4** nell'implementazione PyTorch) introducendo le **Convoluzioni Dilatate** (*Atrous Convolutions*). Modificando il parametro di dilatazione (rispettivamente a 2 e a 4), la rete espande il *Receptive Field* dei filtri convoluzionali senza ridurre ulteriormente la dimensionalità dell'immagine. Il tensore di *feature* risultante mantiene così una risoluzione pari a $1/8$ dell'input originale, offrendo un compromesso ottimale tra estrazione semantica profonda e preservazione del dettaglio spaziale.

3.2 Il Pyramid Pooling Module (PPM)

Il tensore di *feature* estratto dal backbone contiene informazioni fortemente localizzate, ma manca di una visione globale strutturata. Questo limite viene superato dal **Pyramid Pooling Module (PPM)**, il vero cuore dell'architettura.

Il PPM prende la *feature map* della ResNet e la processa parallelamente attraverso quattro rami di *Adaptive Average Pooling*, creando rappresentazioni della scena a diverse scale (i cosiddetti *bins*). Nell'implementazione sviluppata, i *bins* utilizzati hanno dimensioni 1×1 , 2×2 , 3×3 e 6×6 :

- Il bin 1×1 condensa l'intera *feature map* in un singolo vettore, estraendo il contesto globale assoluto dell'immagine.
- I *bins* successivi suddividono l'immagine in sottomatrici (es. 6×6 crea 36 sotto-regioni), estraendo contesti locali a grana via via più fine.

L'output di ciascun ramo passa attraverso una convoluzione 1×1 per ridurre il numero di canali, alleggerendo il carico computazionale. Successivamente,

queste quattro mappe multi-scala vengono riportate alla risoluzione della *feature map* originale tramite un campionamento bilineare (*Bilinear Upsampling*) e, infine, concatenate lungo l'asse dei canali con l'output diretto della ResNet-50. Il risultato è una *feature map* arricchita che unisce il dettaglio locale all'informazione contestuale globale. Un layer convoluzionale finale (classificatore) proietta questo tensore nello spazio delle classi desiderate, generando un output di profondità pari a $C = 25$ (24 classi oggetto + background).

3.3 Funzione di Costo: Cross Entropy Loss e pesi "Inverse Frequency"

L'addestramento supervisionato del segmentatore richiede una metrica differenziabile che misuri la distanza tra la mappa di probabilità predetta dal modello e la maschera *ground truth*. A tal fine, viene impiegata la **Cross-Entropy Loss (CE)** applicata a livello di singolo pixel. Matematicamente, per un pixel i , la *loss* è definita come:

$$\mathcal{L}_{CE} = - \sum_{c=0}^{C-1} y_{i,c} \log(p_{i,c})$$

dove $y_{i,c}$ è un indicatore binario (1 se la classe vera del pixel è c , altrimenti 0) e $p_{i,c}$ è la probabilità predetta dalla rete (tramite funzione Softmax) che il pixel appartenga alla classe c .

Data l'estrema predominanza della classe "Background", la Cross-Entropy è stata matematicamente pesata (*Weighted Cross-Entropy*). L'approccio scelto è l'**Inverse Frequency Weighting**: il modello scansiona un sottoinsieme rappresentativo del dataset sorgente calcolando un istogramma globale delle frequenze dei pixel per ogni classe (f_c). I pesi w_c sono calcolati tramite una funzione di decadimento logaritmico:

$$w_c = \frac{1}{\log(1.02 + f_c)}$$

Questa formulazione evita divisioni per zero ed "ammorbidisce" le penalità estreme. Successivamente, per decurtare l'impatto preponderante del background, il peso della Classe 0 viene esplicitamente abbattuto di un ordine di grandezza ($w_0 = w_0 \times 0.1$). Infine, i pesi vengono normalizzati affinché la loro somma sia proporzionale al numero totale di classi.

Passando questo vettore di pesi al tensore della *Loss Function*, i gradienti propagati all'indietro durante gli errori sulle classi rare risultano artificialmente amplificati, permettendo al modello di mappare correttamente i confini degli oggetti minoritari fin dalle prime epoche di training.

4 Traduzione di Immagini e Generative Adversarial Networks (GAN)

Per risolvere il problema dell'adattamento di dominio non supervisionato (*Un-supervised Domain Adaptation*, UDA), l'approccio standard prevede l'utilizzo di tecniche di *Image-to-Image Translation*. L'obiettivo è mappare le immagini dal dominio sorgente (Sintetico) al dominio target (Reale) alterandone lo stile visivo (illuminazione, texture, rumore del sensore) ma mantenendone inalterato il contenuto semantico (forma e posizione degli oggetti), affinché le maschere di segmentazione sintetiche originali rimangano valide anche sull'immagine tradotta.

4.1 Basi delle GAN: Generatore e Discriminatore

La tecnologia fondante è rappresentata dalle **Generative Adversarial Networks (GAN)**. Una GAN è composta da due reti neurali addestrate simultaneamente in un gioco a somma zero (approccio *minimax*):

- Il **Generatore** (G): ha il compito di prendere un'immagine sintetica x e generare un'immagine fittizia $G(x)$ visivamente indistinguibile da un'immagine reale.
- Il **Discriminatore** (D): agisce come un classificatore binario. Riceve in input immagini reali y e immagini generate $G(x)$ imparando a distinguerle.

Nell'implementazione sviluppata, il Generatore è basato su un'architettura ResNet composta da convoluzioni di *downsampling*, 9 blocchi residuali centrali e convoluzioni trasposte per l'*upsampling*. Il Discriminatore implementato è un **PatchGAN** (definita come `NLayerDiscriminator`), che valuta la "realisticità" di singole patch locali $N \times N$, incentivando la nitidezza delle texture. Per garantire maggiore stabilità, è stata utilizzata la **Least Squares GAN (LSGAN)**, che sostituisce la funzione di costo logaritmica standard con l'Errore Quadratico Medio (MSE).

4.2 Limiti della CycleGAN e introduzione all'apprendimento contrastivo (CUT)

Nel paper di riferimento, gli autori affrontano la traduzione Sintetico-Reale utilizzando la **CycleGAN**. Poiché i dataset non sono appaiati (*unpaired*), la CycleGAN impone un vincolo di *Cycle-Consistency* ($F(G(x)) \approx x$). Tuttavia, tale approccio presenta limiti computazionali:

1. Richiede l'allocazione di due Generatori e due Discriminatori, raddoppiando l'impatto sulla VRAM.
2. È incline a fenomeni di *steganografia visiva*, dove il Generatore nasconde informazioni nel rumore pur di soddisfare la *loss* ciclica.

3. Rende quasi impossibile l'addestramento congiunto (*online joint-training*) con reti di segmentazione pesanti come la PSPNet.

Per superare queste barriere, il progetto adotta un approccio basato sull'**Apprendimento Contrastivo** (*Contrastive Learning*), ispirato all'architettura CUT (*Contrastive Unpaired Translation*). L'idea è abbandonare la traduzione ciclica e preservare il contenuto massimizzando l'*Informazione Reciproca* tra le patch di input e di output.

4.3 Architettura OS-DCLGAN (One-Sided Domain Adaptation)

L'implementazione finale è denominata **OS-DCLGAN** (*One-Sided Dual Contrastive Learning GAN*). A differenza della CycleGAN, utilizza un solo Generatore e un solo Discriminatore. Il mantenimento del contenuto strutturale è garantito in un singolo passaggio (*forward pass*): il Generatore agisce sia da traduttore che da estrattore di *feature*.

Selezionando specifici layer intermedi dell'architettura ResNet, viene estratta una piramide di *feature map* inviate al modulo contrastivo. Questa snellezza architettonica ha reso possibile l'*end-to-end joint-training*: mentre il Generatore adatta il dominio visivo, la PSPNet riceve immediatamente l'immagine tradotta per calcolare i gradienti di segmentazione.

4.4 PatchNCE Loss e modulo di attenzione spaziale (CBAM)

Il cuore dell'apprendimento contrastivo è la **PatchNCE Loss** (*Noise Contrastive Estimation*). Per una specifica patch (*Query*) nell'output, si definiscono:

- **Esempio Positivo:** La patch estratta dall'immagine di input nella stessa coordinata.
- **Esempi Negativi:** Tutte le altre patch estratte da posizioni differenti dell'input.

L'equazione, governata dal parametro di temperatura τ , è:

$$\ell(v, v^+, v^-) = -\log \left[\frac{\exp(v \cdot v^+ / \tau)}{\exp(v \cdot v^+ / \tau) + \sum_{n=1}^N \exp(v \cdot v_n^- / \tau)} \right]$$

Per ottimizzare il campionamento delle patch, è stato integrato un **Convolutional Block Attention Module (CBAM)**. Questo modulo inferisce mappe di attenzione spaziale e di canale, forzando la *PatchNCE Loss* a concentrarsi sui bordi e sulle texture ad alta frequenza anziché su sfondi omogenei. Tale sinergia assicura che, durante la traduzione, i contorni degli oggetti culturali rimangano intatti e perfettamente allineati con le maschere *ground truth*, fornendo alla PSPNet un input ibrido di altissima fedeltà.

5 Metriche di Valutazione per la Segmentazione

La valutazione delle performance di un modello di segmentazione semantica richiede metriche specifiche. A causa dello sbilanciamento delle classi (dove il *background* o grandi elementi strutturali dominano la scena), misurare unicamente la percentuale globale di predizioni corrette risulta fuorviante.

Nelle formulazioni matematiche a seguire, assumeremo:

- C : numero totale di classi (nel nostro caso $C = 25$, ignorando i pixel invalidi).
- n_{ij} : numero di pixel appartenenti alla classe vera i e predetti come classe j .
- n_{ii} : numero di pixel classificati correttamente (*Veri Positivi* per la classe i).
- $t_i = \sum_j n_{ij}$: numero totale di pixel appartenenti alla classe i nel *ground truth*.

5.1 Intersection over Union (IoU) e Mean IoU (mIoU)

L'**Intersection over Union (IoU)**, nota anche come indice di Jaccard, misura la sovrapposizione percentuale tra l'area predetta e l'area reale. È definita come il rapporto tra l'intersezione e l'unione dei due insiemi di pixel:

$$IoU_i = \frac{n_{ii}}{t_i + \sum_j n_{ji} - n_{ii}} = \frac{TP}{TP + FP + FN}$$

Questa metrica penalizza equamente sia i *Falsi Positivi* che i *Falsi Negativi*. Per valutare la bontà globale, si calcola la **Mean IoU (mIoU)**, media aritmetica dell'IoU per ciascuna classe:

$$mIoU = \frac{1}{C} \sum_{i=0}^{C-1} IoU_i$$

5.2 Pixel Accuracy (PA) e Mean Pixel Accuracy (mPA)

L'**Overall Pixel Accuracy (PA)** rappresenta la percentuale globale di pixel classificati correttamente rispetto al totale dei pixel validi:

$$PA = \frac{\sum_{i=0}^{C-1} n_{ii}}{\sum_{i=0}^{C-1} t_i}$$

Sebbene la PA tenda a sovrastimare le performance, fornisce un'indicazione macroscopica della fedeltà visiva. Per mitigare il *bias* verso le classi maggioritarie, si calcola la **Mean Pixel Accuracy (mPA)**:

$$mPA = \frac{1}{C} \sum_{i=0}^{C-1} \frac{n_{ii}}{t_i}$$

5.3 Frequency Weighted Average Accuracy (FWAVACC)

Per confrontare rigorosamente i risultati ottenuti dall'architettura OS-DCLGAN con quelli della letteratura, è stata integrata la **FWAVACC** (*Frequency Weighted Average Intersection over Union*). Questa metrica pesa l'IoU di ciascuna classe in base alla frequenza con cui essa appare nel *ground truth*:

$$FWAVACC = \frac{1}{\sum_k t_k} \sum_{i=0}^{C-1} \left(t_i \cdot \frac{n_{ii}}{t_i + \sum_j n_{ji} - n_{ii}} \right)$$

5.4 La Matrice di Confusione

A livello implementativo, il calcolo delle metriche è stato ottimizzato tramite la **Matrice di Confusione**. Durante l'inferenza, i tensori dell'output e le maschere reali vengono appiattiti (*flattening*) dopo aver escluso i pixel invalidi (etichettati come 255):

```
mask = (label_flat != 255)
all_preds.extend(pred_flat[mask])
all_targets.extend(label_flat[mask])
```

I vettori risultanti popolano una matrice quadrata $C \times C$, dove le righe indicano il *ground truth* e le colonne le predizioni. Estrarre la traccia (n_{ii}) e sommare lungo gli assi permette di derivare in tempo reale l'IoU, la mPA, la PA e la FWAVACC, garantendo un'analisi precisa su tutto il volume del dataset di validazione.

6 Segmentazione di Oggetti Culturali: Il Dataset EGO-CH-OBJ-SEG

6.1 Analisi esplorativa del Dataset (Sintetico e Reale)

Il dataset utilizzato, EGO-CH-OBJ-SEG, rappresenta una risorsa fondamentale per la segmentazione in ambienti museali. La sua peculiarità risiede nella natura duale:

- **Dominio Sintetico:** Composto da scene renderizzate in cui ogni oggetto è perfettamente isolato e annotato. Fornisce un numero elevato di campioni, ma soffre della mancanza di realismo visivo (assenza di imperfezioni fisiche, illuminazione ideale).
- **Dominio Reale:** Composto da sequenze video acquisite da dispositivi indossabili (*egocentric view*). Queste immagini contengono le sfide tipiche del mondo reale: sfocature da movimento, variazioni drastiche di illuminazione e rumore del sensore che rende la segmentazione basata su modelli sintetici estremamente difficile.

L'analisi esplorativa ha evidenziato una distribuzione a "coda lunga": un numero ristretto di classi (pareti, pavimenti, basi espositive) occupa la maggior parte della superficie dell'immagine, mentre gli artefatti culturali sono distribuiti in modo sparso e occupano regioni limitate dello spazio.

6.2 Data Engineering: Risoluzione dell' "Exact RGB Matching"

Una delle sfide ingegneristiche più critiche è stata l'inconsistenza nel formato delle maschere di segmentazione. Le annotazioni originali non erano memorizzate solo come immagini indicizzate (formato P o L), ma spesso come immagini RGB (24-bit). In un'immagine RGB, il valore del pixel è una tripla di colori (es. $[0, 117, 21]$), il che impediva il caricamento diretto nei tensori delle funzioni di *loss*.

Per risolvere il problema, è stato sviluppato un algoritmo di decodifica vettorializzata basato sulla distanza euclidea:

1. **Mappatura della Palette:** È stata ricostruita la *colormap* ufficiale, associando ogni tripla RGB all'indice di classe corrispondente.
2. **Calcolo della Distanza Vettorializzata:** Sfruttando le operazioni di *broadcasting* di NumPy, abbiamo calcolato la distanza tra il colore del pixel e tutte le 25 classi simultaneamente:

$$distanza = \sqrt{\sum (pixel_{RGB} - colormap_{classe})^2}$$

3. **Assegnazione del "Nearest Neighbor":** Ogni pixel viene assegnato alla classe il cui colore nella palette è il più vicino in termini di distanza euclidea.
4. **Filtro di tolleranza:** Per evitare errori di classificazione su artefatti di compressione, se la distanza euclidea al quadrato supera la soglia di 1500, il pixel viene contrassegnato come 255 (etichetta di ignoranza per la *loss*).

6.3 Tecniche di Data Augmentation e Preprocessing

Per massimizzare la robustezza della rete e prevenire l'*overfitting*, la pipeline di caricamento include:

- **Mirroring Spaziale (*Horizontal Flip*):** Specchiatura casuale (probabilità 50%) per aumentare la variabilità delle pose degli oggetti.
- **Color Jittering:** Applicazione di variazioni casuali a luminosità, contrasto, saturazione e tinta (*hue*) per simulare le variazioni di illuminazione tra diverse sale museali.
- **Normalizzazione:** Poiché il Generatore utilizza una funzione di attivazione *Tanh*, le immagini sono state normalizzate nell'intervallo $[-1, 1]$ sottraendo la media e dividendo per la deviazione standard, garantendo coerenza con le aspettative del modello.

7 Implementazione e Metodologia di Training

L'addestramento di modelli complessi che integrano reti generative e segmentatori semantici richiede un'architettura di *training* attentamente orchestrata. In questo capitolo verranno descritte le metodologie implementative adottate per garantire la convergenza del modello, dalla struttura del *joint-training* fino agli accorgimenti per la stabilità numerica.

7.1 Architettura di Joint-Training (Training congiunto GAN + PSPNet)

Il nucleo innovativo del progetto risiede nel *Joint-Training end-to-end* tra l'architettura OS-DCLGAN (Generatore e Discriminatore) e la PSPNet. Invece di addestrare i due modelli in fasi separate (prima la traduzione di stile, poi la segmentazione), i pesi vengono aggiornati simultaneamente ad ogni *batch*.

Durante la fase di *forward*, il flusso dei dati segue un percorso preciso:

1. **Generazione:** Un'immagine sintetica (con la sua maschera perfetta) viene passata al Generatore, che produce un'immagine tradotta (stile reale, contenuto sintetico).
2. **Segmentazione (Fake & Real):** L'immagine tradotta viene passata alla PSPNet per calcolare la *loss* di segmentazione. Parallelamente, anche l'immagine sintetica originale (e l'immagine reale, nel caso supervisionato) attraversa il segmentatore.
3. **Discriminazione:** L'immagine tradotta e un'immagine reale *unpaired* vengono valutate dal Discriminatore per il calcolo della *loss* avversaria.

In questo modo, la PSPNet agisce implicitamente come un regolarizzatore per il Generatore: se quest'ultimo distorce la geometria dell'oggetto durante la traduzione, la PSPNet fallirà la segmentazione, propagando gradienti elevati che costringeranno il Generatore a preservare meglio le forme. L'ottimizzazione è gestita da tre ottimizzatori **Adam** indipendenti, modulati da uno *scheduler* **CosineAnnealingLR**.

7.2 Stabilità Numerica: Risoluzione dei NaN, Gradient Clipping e Precisione FP32

Come introdotto nel Capitolo 2, l'addestramento combinato GAN-PSPNet è intrinsecamente instabile. Nelle prime fasi di sviluppo, il modello andava frequentemente incontro all'esplosione dei gradienti, culminando nella corruzione dei pesi (valori **NaN** – *Not a Number*).

L'analisi del problema ha rivelato due cause principali: i calcoli logaritmici della *PatchNCE Loss* contrastiva e l'uso della *Mixed Precision* (FP16). Per ripristinare la stabilità, sono state implementate le seguenti soluzioni ingegneristiche:

- **Disattivazione parziale dell'AMP (*Automatic Mixed Precision*):** Sebbene l'FP16 dimezzi l'uso della VRAM, i valori estremi generati dalla *loss* contrastiva causavano *underflow* numerico. Forzare l'uso della precisione singola (FP32) per i layer critici ha eliminato gli errori di approssimazione.
- **Gradient Clipping:** Prima di ogni aggiornamento dei pesi (`optimizer.step()`), i gradienti vengono limitati tramite la funzione `torch.nn.utils.clip_grad_norm_` con un `max_norm=10.0`. Questo impedisce che singoli *batch* anomali (es. un'immagine completamente nera generata per errore) distruggano quanto appreso nelle epoche precedenti.
- **Controlli di Integrità (*Sanity Checks*):** È stata introdotta una logica condizionale rigorosa (`if not torch.isnan(loss)`) che salta l'aggiornamento dei pesi qualora la funzione di costo restituisca valori infiniti o indefiniti, fungendo da "salvavita" per l'intero *training*.

7.3 La strategia di "Warm-up" per la salvaguardia del Segmentatore

Per contrastare il fenomeno del *Cold Start* e il conseguente *Background Collapse* (descritto nella Sezione 2.3), è stata ideata e integrata nello script di *training* una strategia di **Warm-up Asimmetrico**.

Nelle primissime epoche (es. `warmup_epochs = 3`), il Generatore produce inevitabilmente output caotici e rumorosi. Se la PSPNet fosse addestrata da subito su queste immagini corrotte, i suoi pesi – inizialmente pre-addestrati in modo robusto su ImageNet – collaserebbero istantaneamente verso la banale predizione del *background*.

La strategia implementata prevede che, durante il periodo di *warm-up*, la *loss* di segmentazione venga calcolata **esclusivamente sulle immagini sintetiche originali**, isolando la PSPNet dagli esperimenti fallimentari del Generatore:

```
if epoch < args.warmup_epochs:
    # Warm-up: La PSPNet impara solo dai dati sintetici originali
    loss_seg = loss_seg_real
else:
    # Domain Adaptation: La PSPNet usa anche i dati generati (Fake)
    loss_seg = (loss_seg_fake + loss_seg_real) * 0.5
```

Terminato il *warm-up*, il Generatore è ormai in grado di produrre immagini con uno stile reale quantomeno abbozzato e, soprattutto, con i contorni degli oggetti intatti. A questo punto, il flusso di dati ibridi viene "aperto" verso la PSPNet, che può così beneficiare della *Domain Adaptation* partendo da una base di conoscenza già solida e stabile, garantendo una convergenza sicura e progressiva verso percentuali di *Accuracy* elevate.

8 Esperimenti e Risultati

In questo capitolo vengono presentati gli esperimenti condotti per valutare l'efficacia dell'architettura OS-DCLGAN combinata con la PSPNet. L'analisi si concentra sulla progressione delle performance passando da un regime completamente non supervisionato (UDA) a regimi semi e pienamente supervisionati.

8.1 Setup dell'Infrastruttura e Limiti Computazionali

Date le elevate richieste computazionali del *Joint-Training*, gli esperimenti sono stati condotti su un cluster HPC (High Performance Computing) messo a disposizione dal Dipartimento di Matematica e Informatica dell'Università degli Studi di Catania. L'ambiente di esecuzione è stato standardizzato tramite **App-tainer**, garantendo l'incapsulamento delle dipendenze e l'indipendenza dal sistema ospite.

Ai fini dell'analisi, sono stati selezionati tre scenari rappresentativi in grado di coprire l'intero spettro della *Domain Adaptation*: **0% (baseline UDA)**, **50% (SSDA)** e **100% (Fully-Supervised)** di dati reali. Gli esperimenti sono stati eseguiti in sequenza, vincolando ciascun processo a una singola GPU con circa 22 GB di VRAM allocata.

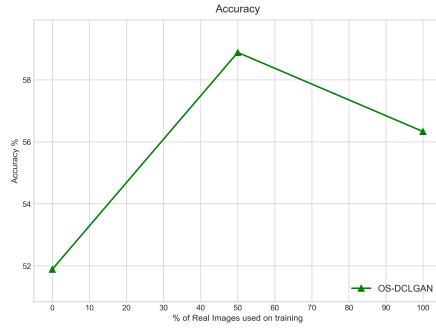
Un ostacolo ingegneristico rilevante è stato dettato dalle politiche di utilizzo del nodo di calcolo, che imponevano un *timeout* massimo ineludibile di 12 ore per singolo *job*. Mentre l'addestramento della *baseline* allo 0% è riuscito a concludersi con successo all'interno della finestra temporale concessa, l'elaborazione dei modelli al 50% e al 100% (a causa del carico computazionale aggiuntivo dovuto all'integrazione del dataset reale) non riusciva a terminare nelle 12 ore previste. Per superare questo limite infrastrutturale, è stata progettata e integrata nel codice una logica di *checkpointing* e recupero dello stato: il sistema salva i pesi parziali delle reti (GAN e PSPNet) al termine di ogni epoca, consentendo al riavvio del *job* di effettuare il *resume* dell'addestramento, riprendendo esattamente dal punto in cui il processo era stato interrotto dal cluster.

8.2 Analisi dei Risultati

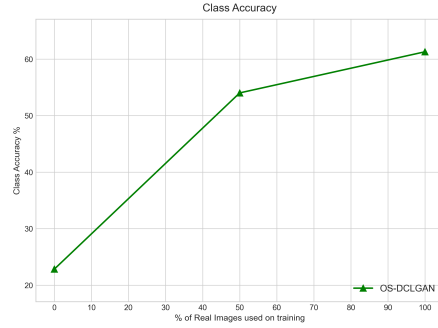
Le performance dei tre modelli sono state valutate sul *test set* reale calcolando le quattro metriche di riferimento (PA, mPA, mIoU, FWAVACC) discusse nel Capitolo 5. In linea con la metodologia della letteratura di riferimento, l'analisi procede valutando inizialmente i modelli con accesso alle etichette reali (100% e 50%), per poi culminare nell'analisi del regime puramente non supervisionato (0%). Questo approccio permette di dimostrare come, pur riducendo drasticamente (o azzerando) i dati reali, si ottengano risultati di segmentazione del tutto accettabili, validando l'obiettivo cardine della ricerca: abbattere i costi di annotazione manuale. I risultati quantitativi sono riassunti nella Tabella 1 e visualizzati graficamente nella Figura 1.

Table 1: Confronto delle performance di segmentazione al variare della percentuale di dati reali supervisionati. I valori migliori per ciascuna metrica sono evidenziati in grassetto.

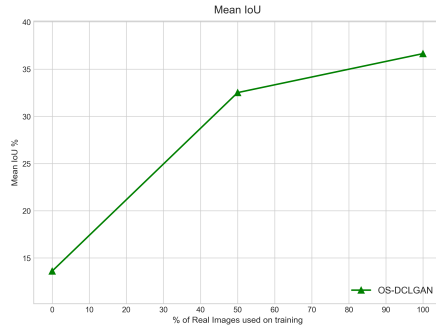
Modello (Dati Reali)	PA (%)	mPA (%)	mIoU (%)	FWAVACC (%)
OS-DCLGAN (100%)	56.33	61.29	36.64	41.06
OS-DCLGAN (50%)	58.88	54.01	32.51	42.31
OS-DCLGAN (0%)	51.89	22.83	13.61	37.07



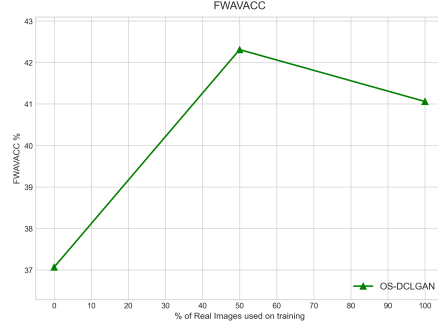
(a) Pixel Accuracy (PA).



(b) Mean Pixel Accuracy (mPA).



(c) Mean Intersection over Union (mIoU).



(d) Frequency Weighted IoU (FWAVACC).

Figure 1: Andamento delle metriche di segmentazione su test set reale al variare della percentuale di dati reali utilizzati in fase di addestramento. Le curve evidenziano il comportamento non lineare delle performance all'aumentare della supervisione.

Come emerge chiaramente dall'andamento delle curve nei grafici in Figura 1, il rapporto tra la quantità di dati reali e le performance del modello non è strettamente lineare, confermando la solidità dell'approccio proposto. Mentre le metriche non pesate (mIoU e mPA) mostrano una crescita progressiva (con un

andamento quasi asintotico verso il 100%), la Pixel Accuracy e la FWAVACC rivelano un netto comportamento a campana. Questa rappresentazione visiva anticipa e giustifica le dinamiche specifiche discusse nei paragrafi seguenti, dimostrando come un’etichettatura totale non garantisca necessariamente un incremento assoluto in ogni metrica.

8.2.1 Domain Adaptation Fully-Supervised (100% Dati Reali)

L’esperimento con il 100% di dati reali rappresenta l’*upper bound* (limite superiore) delle capacità del modello. Sfruttando l’intero *ground truth* reale a disposizione, la rete raggiunge il suo potenziale massimo sulle metriche non pesate: **la mPA tocca il 61.29% e la mIoU il 36.64%** (Fig. 1b e 1c). In questo scenario, il modello sviluppa un apprendimento profondo e accurato delle classi minoritarie e degli artefatti di piccole dimensioni (es. teche, cartelli, estintori), districando con efficacia le complessità semantiche dei singoli oggetti. Tuttavia, è fondamentale sottolineare il notevole costo computazionale di questo approccio: l’elaborazione simultanea del dataset sintetico e dell’intero dataset reale ha richiesto molteplici riavvii tramite la logica di *checkpointing*, rendendo possibile l’addestramento per sole 30 epoche (oltre alla fase di *warm-up*) prima di saturare le ripetute finestre di esecuzione da 12 ore imposte dal nodo HPC.

8.2.2 Domain Adaptation Semi-Supervisionata (50% Dati Reali)

Dimezzare lo sforzo di annotazione manuale (utilizzando solo il 50% dei dati reali) non si traduce in un degrado fatale delle prestazioni; al contrario, produce una dinamica eccellente. In questo scenario si registra infatti il **picco assoluto della Pixel Accuracy (58.88%) e della FWAVACC (42.31%)** (Fig. 1a e 1d). Poiché queste due metriche sono pesate sulla frequenza dei pixel, il risultato indica che un equilibrio ibrido fornisce il punto di ottimo per il riconoscimento delle macro-classi strutturali (es. *Background*, Muri, Pavimenti) che dominano visivamente l’ambiente museale. Anche in questo caso, come per il modello pienamente supervisionato, l’addestramento si è fermato a 30 epoche a causa delle stringenti tempistiche di calcolo.

8.2.3 Unsupervised Domain Adaptation (0% Dati Reali)

Il risultato più rilevante dell’intera sperimentazione, coerentemente con la tesi promossa dal paper di riferimento, risiede nel regime puramente non supervisionato. Lo scopo ultimo della *Domain Adaptation* è proprio quello di dimostrare che è possibile ottenere risultati di segmentazione accettabili azzerando completamente l’uso di dati reali etichettati.

A differenza dei casi supervisionati, l’assenza del dataset reale ha alleggerito drasticamente il carico computazionale. Questa maggiore snellezza in fase di *training* ha permesso al modello di addestrarsi ininterrottamente per ben **80 epoche** all’interno di una singola esecuzione di 12 ore sul cluster. Sebbene le metriche più severe (come la mIoU al 13.61%) riflettano una difficoltà nel definire

i contorni esatti delle classi rare senza alcun segnale reale, la **Pixel Accuracy globale supera il 51%**. Questo dato attesta che l'apprendimento contrastivo (*PatchNCE Loss*) permette al Generatore di tradurre le *texture* di base in modo sufficientemente accurato da ingannare positivamente il segmentatore. Il modello è dunque capace di astrarre forme complesse affidandosi esclusivamente a dati sintetici, confermando il successo dell'approccio proposto.

8.3 Considerazioni sull'efficienza (OS-DCLGAN vs CycleGAN)

Oltre ai risultati numerici, l'esperimento valida il successo ingegneristico dell'architettura OS-DCLGAN. Laddove l'approccio classico (CycleGAN) si scontra con limiti insormontabili di VRAM a causa della doppia istanziazione di generatori e discriminatori, il modello *One-Sided* proposto ha permesso l'esecuzione del *Joint-Training end-to-end* all'interno di una singola GPU standard, completando l'addestramento e producendo output di alta qualità geometrica (esenti da artefatti steganografici) prima del termine imposto dal cluster.

9 Conclusioni

9.1 Considerazioni finali sull'efficacia dell'approccio

Il presente progetto ha affrontato una sfida complessa nell'ambito della *Computer Vision* applicata ai Beni Culturali: colmare il *Domain Gap* tra immagini sintetiche e acquisizioni reali in ottica egocentrica, garantendo al contempo un'efficienza computazionale sostenibile.

L'obiettivo è stato ampiamente raggiunto grazie alla progettazione e all'implementazione di una pipeline di *Joint-Training end-to-end* che ha integrato un modello generativo all'avanguardia (OS-DCLGAN) con un segmentatore robusto al contesto (PSPNet). Il passaggio dalla tradizionale CycleGAN all'apprendimento contrastivo (*PatchNCE Loss*) ha rappresentato un salto di qualità decisivo: ha dimezzato i requisiti di memoria VRAM, ha eliminato gli artefatti di steganografia visiva e ha permesso l'ottimizzazione simultanea delle due reti su una singola GPU.

Oltre al design architetturale, le scelte di *Data Engineering* e gestione numerica hanno giocato un ruolo fondamentale:

- L'implementazione dell'*Exact RGB Matching* tramite distanza euclidea vettorializzata ha risolto il problema dell'inconsistenza del formato delle annotazioni. Mappando in modo esatto i colori RGB delle maschere ai rispettivi indici di classe della *colormap*, è stato possibile generare tensori direttamente elaborabili dalle funzioni di costo.
- L'introduzione di un **Warm-up asimmetrico** per la PSPNet e la gestione dei gradienti (*Gradient Clipping* e *sanity checks* sui valori NaN)

hanno disinnescato il problema del *Cold Start* e del *Background Collapse*, garantendo una convergenza stabile fin dalle prime epoche.

I risultati sperimentali confermano l'ipotesi di ricerca: è possibile ridurre drasticamente i dati reali etichettati mantenendo performance di segmentazione accettabili. Il regime non supervisionato (0%) dimostra la fattibilità di un sistema senza costi di annotazione. L'esperimento al 50% rappresenta il compromesso ideale costi-benefici, massimizzando Pixel Accuracy e FWAVACC. Infine, sebbene il 100% dei dati definisca l'*upper bound* (picco di mIoU), l'analisi prova che un'etichettatura totale non è strettamente necessaria per un'efficace comprensione dell'ambiente museale.

9.2 Sviluppi e lavori futuri

Alla luce dei risultati ottenuti e dell'impianto metodologico sviluppato, i possibili sviluppi futuri includono le seguenti direzioni di ricerca:

1. **Estensione del Training e Hyperparameter Tuning:** Con l'accesso a risorse di calcolo prolungate, sarebbe possibile addestrare il modello per un numero maggiore di epoche (es. 100-200 epoche) ottimizzando la temperatura τ della *PatchNCE Loss* e i ratei di apprendimento tramite algoritmi di ricerca automatizzata (*Grid Search* o *Bayesian Optimization*), incrementando ulteriormente le metriche di accuratezza.
2. **Ottimizzazione per dispositivi Edge (Real-Time):** L'applicazione naturale di questo modello risiede nei dispositivi indossabili (es. visori per la Realtà Aumentata). Poiché la PSPNet con *backbone* ResNet-50 risulta troppo pesante per l'inferenza in tempo reale su hardware *mobile*, sviluppi futuri prevedono la sostituzione dell'estrattore di *feature* con architetture più leggere (come *MobileNetV3*) e l'applicazione di tecniche di *Model Quantization* (da FP32 a INT8) e *Pruning*.
3. **Integrazione di Modelli Foundational (Zero-Shot):** Un'ulteriore frontiera di ricerca prevede l'integrazione di modelli di base pre-addestrati, come il *Segment Anything Model (SAM)*, per assistere o sostituire la GAN nella generazione di pseudo-etichette (*pseudo-labels*) altamente accurate per il dominio target, spingendo la pipeline verso una *Unsupervised Domain Adaptation* dalle performance paragonabili ai modelli puramente supervisionati.

In sintesi, il progetto dimostra che un'attenta ingegnerizzazione del dato, unita a moderne tecniche di apprendimento contrastivo, offre una soluzione elegante, efficiente e altamente scalabile per l'analisi semantica del nostro patrimonio culturale.

References

- [1] F. Rundo, G. L. Banna, C. Spampinato, e S. Palazzo, *Semantic Segmentation of Cultural Heritage Objects in Egocentric Videos*, in Proceedings of the International Conference on Pattern Recognition (ICPR), 2020.
- [2] H. Zhao, J. Shi, X. Qi, X. Wang, e J. Jia, *Pyramid Scene Parsing Network*, in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017.
<https://github.com/hszhao/PSPNet>
- [3] RedPotatoChip, *OSUDL: One-Sided Unsupervised Domain Adaptation via Dual Contrastive Learning*, Repository GitHub, 2023.
<https://github.com/RedPotatoChip/OSUDL>